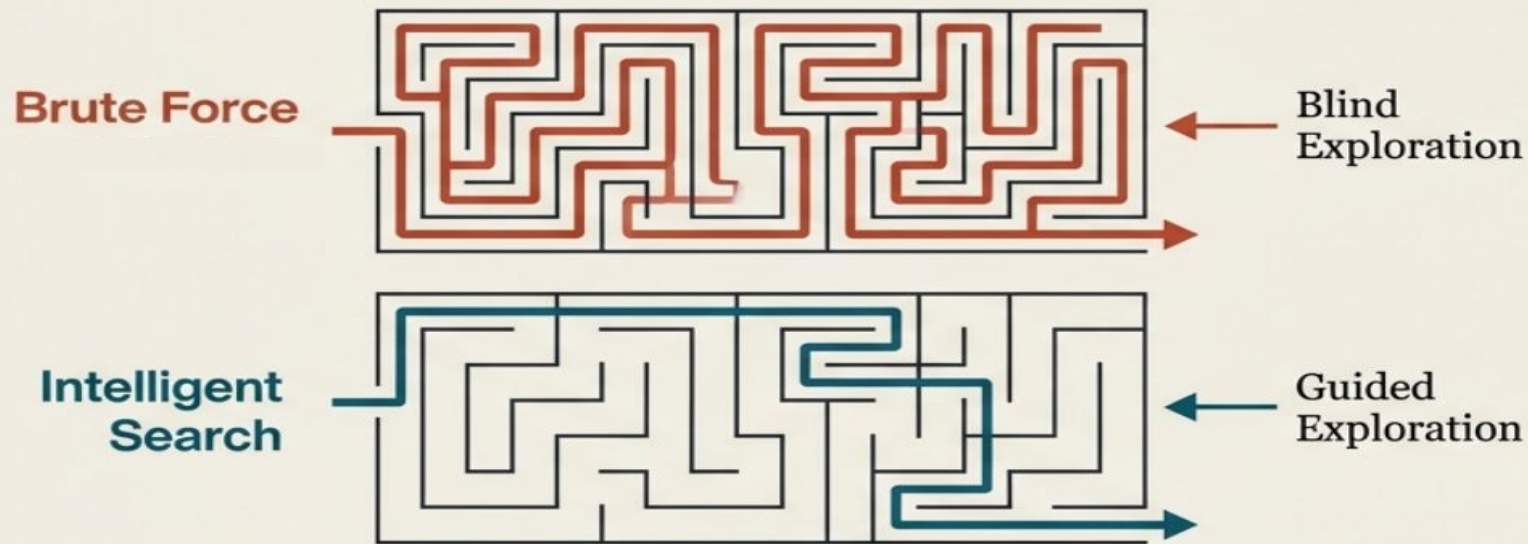


Informed Search

The Efficiency Gap: Working Hard vs. Working Smart

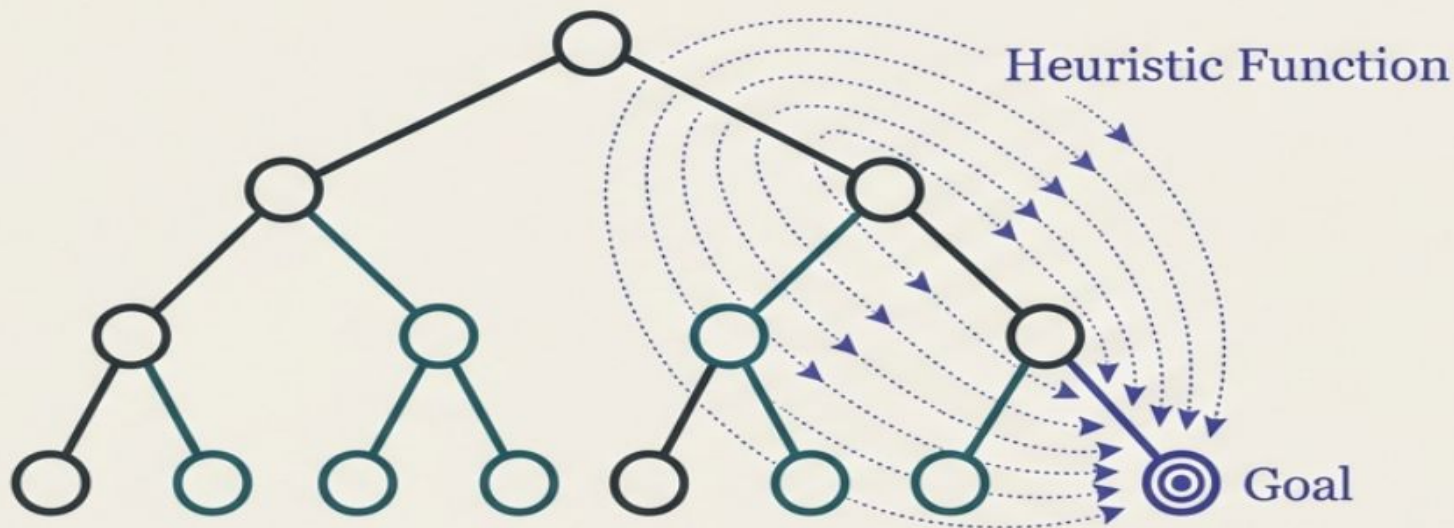
Why raw processing power is no longer enough.



Brute Force approaches problems by exhausting every possibility. It is thorough but computationally expensive. Intelligent Search utilizes guidance to prune the search space, prioritizing paths that are likely to yield a solution.

Heuristics: The Art of the Educated Guess

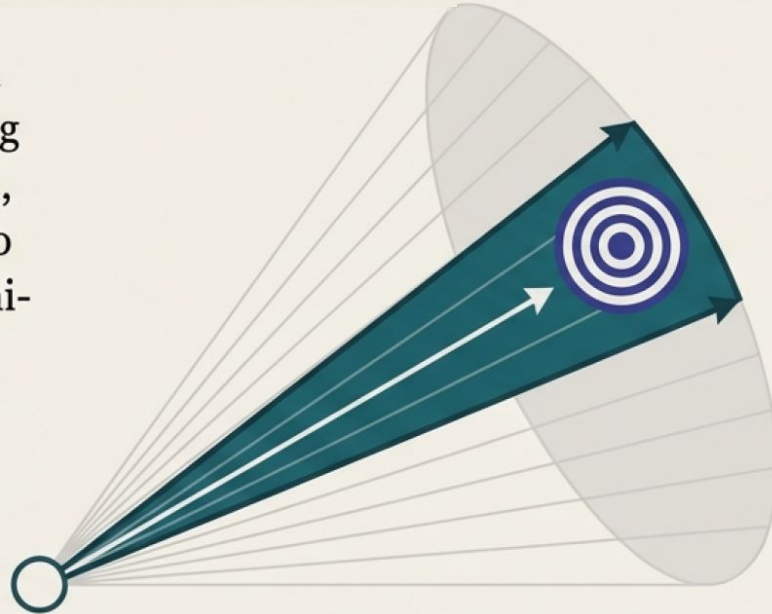
In AI, a Heuristic ($h(n)$) is a function that estimates the cost from the current state to the goal. It serves as computational intuition.



$$h(n) = \text{Estimated distance to goal}$$

Heuristic Search

Heuristics are filters, not guarantees. By estimating proximity to the solution, heuristics allow the AI to prioritize the most promising paths first. A well-designed heuristic acts as a compass in a vast search space.



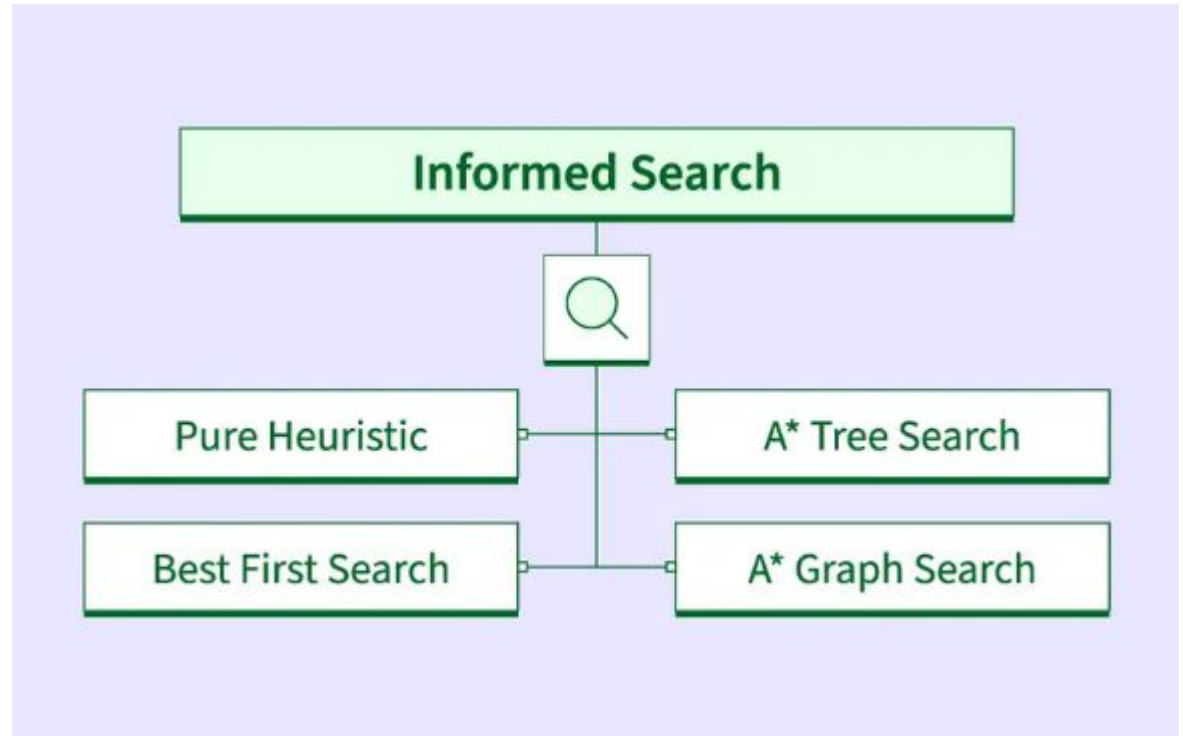
Pruning the Search Space



GPS Logic: When a navigation app calculates your arrival time, it is performing a **heuristic search**. It **estimates** travel time based on distance and current traffic data. It is an **educated prediction**, subject to change, but far superior to driving blindly.

Informed Search

Informed Search = any search that uses heuristics.

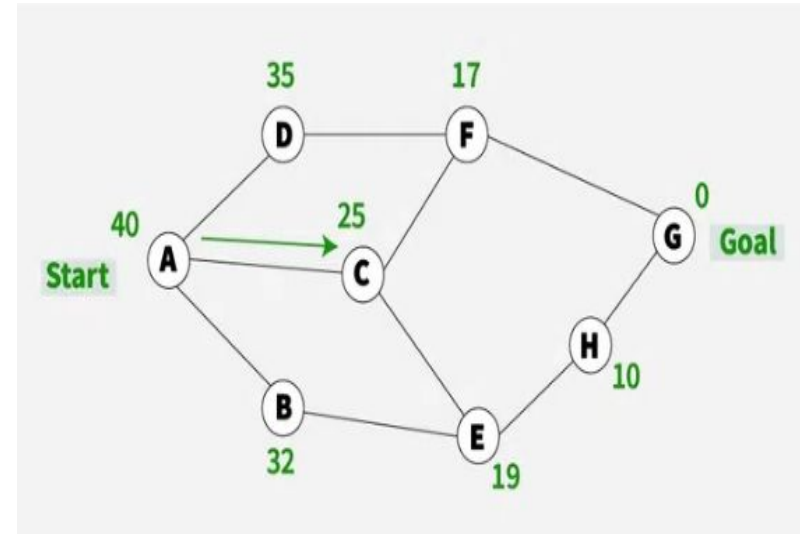


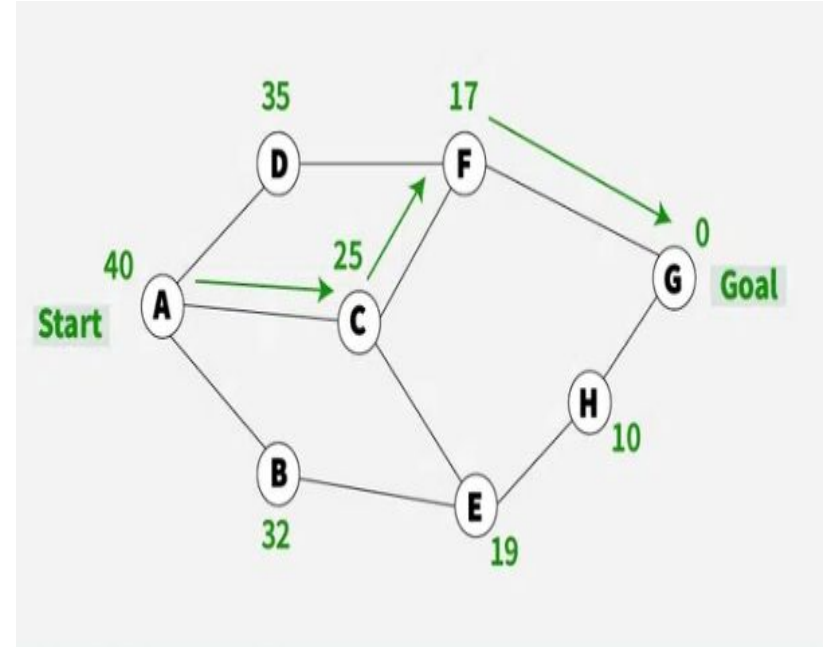
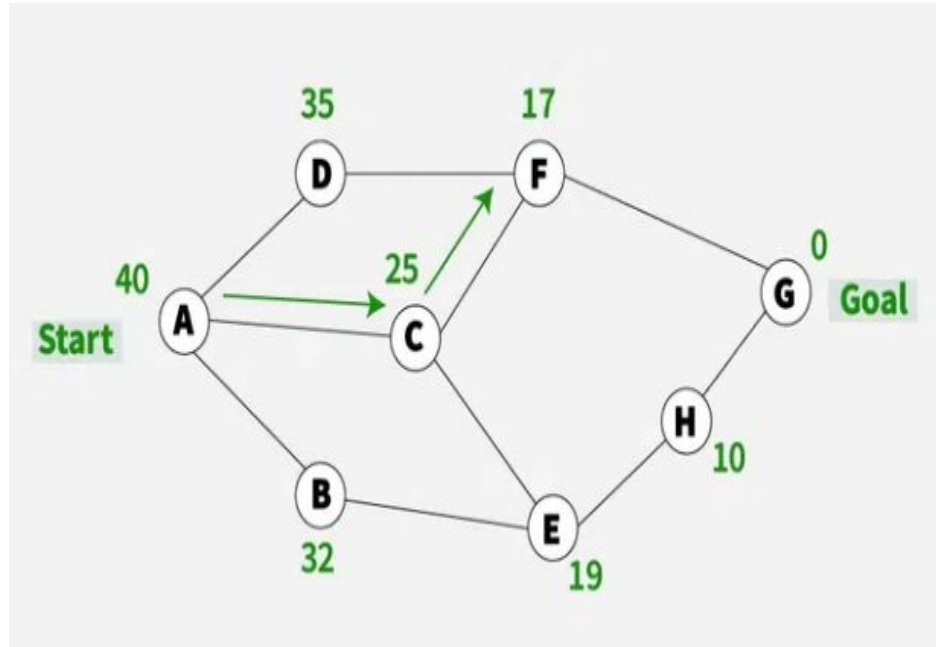
Best-First Search

- Best-First Search is a method that always expands the node that appears closest to the goal using estimated distance.
- Best-First Search is an informed (heuristic-based) search
- Expands the most promising node first
- Uses heuristic function $h(n)$ to estimate closeness to goal
- Evaluation function: $f(n)=h(n)$
- Node with minimum $h(n)$ is selected for expansion

Best-First Search

- Suppose we have to find the path from A to G
- The values in green color represent the **heuristic value of reaching the goal node G from current node**
- We are starting from A, so from A there are direct paths to node B (with a heuristic value of 32), from A to C (with a heuristic value of 25), and from A to D (with a heuristic value of 35).
- So as per the best first search algorithm, **choose the path with the lowest heuristic value**; currently C has the lowest value among the above nodes. **So we will go from A to C.**





So now the goal node G has been reached and the path we will follow is A->C->F->G .

Advantages

- It is more efficient than simple Breadth-First or Depth-First Search because it focuses on the most promising paths.
- The use of heuristics allows for faster goal discovery in many practical applications.
- Low Memory Requirements: Greedy Best-First Search requires only a small amount of memory, making it suitable for applications with limited memory.
- Simple and Easy to Implement: Greedy Best-First Search is a relatively straightforward algorithm, making it easy to implement.

Disadvantages

- It may not always find the optimal solution unless combined with other techniques, such as in A* Search.
- The performance of the search heavily depends on the quality of the heuristic function.
- Local Optima: Greedy Best-First Search can get stuck in local optima, meaning that the path chosen may not be the best possible path.

A* SEARCH ALGORITHM

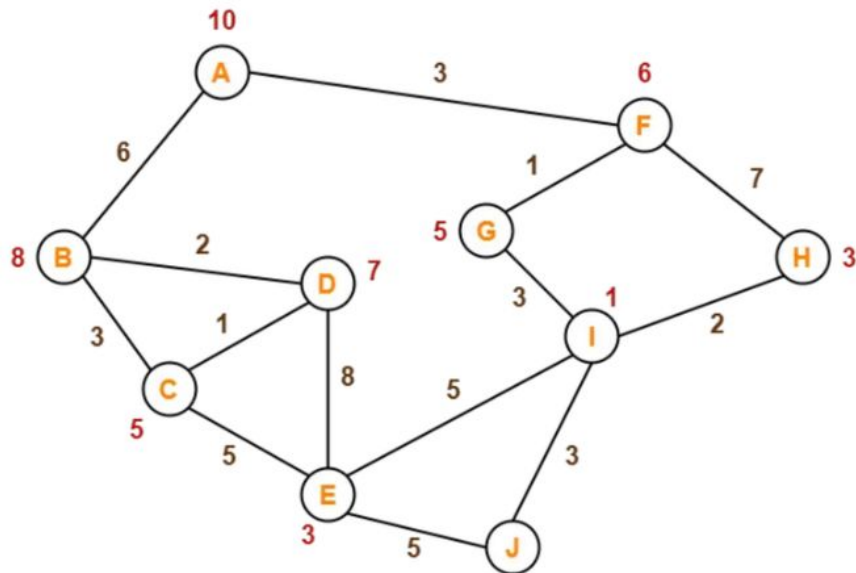
- A* is the most popular informed search algorithm
- **Combines:**
 - Cost so far
 - Estimated cost to goal
- Guarantees optimal solution under conditions
- Widely used in pathfinding and planning

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$: cost from start to node n
- $h(n)$: estimated cost from n to goal
- $f(n)$: total estimated solution cost

A* ALGORITHM



- The numbers written on edges represent the distance between the nodes.
- The numbers written on nodes represent the heuristic value.
- **Find the most cost-effective path to reach from start state A to final state J using A* Algorithm.**

A* ALGORITHM

- **Step-01:**

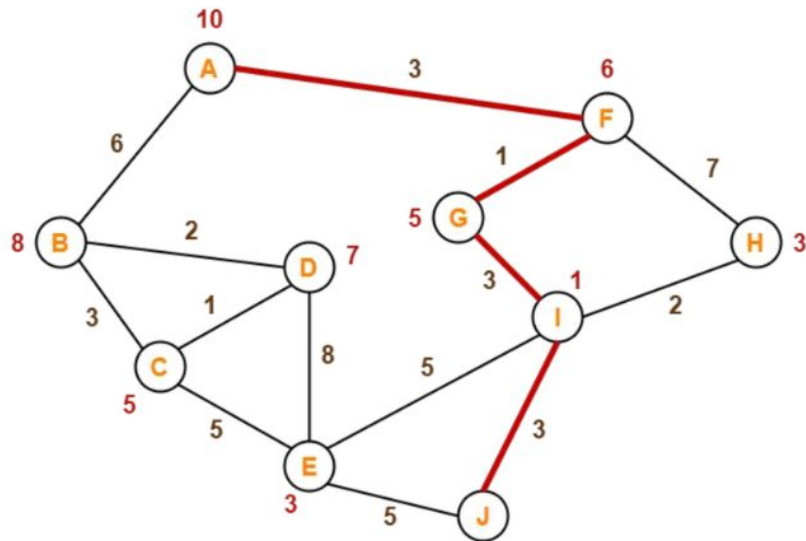
- We start with node A.
- Node B and Node F can be reached from node A.
- A* Algorithm calculates $f(B)$ and $f(F)$.
- $f(B) = 6 + 8 = 14$
- $f(F) = 3 + 6 = 9$
- Since $f(F) < f(B)$, so it decides to go to node F.

- **Path- A $\rightarrow$ F**

- **Step-02:**

- Node G and Node H can be reached from node F.
- A* Algorithm calculates $f(G)$ and $f(H)$.
- $f(G) = (3+1) + 5 = 9$
- $f(H) = (3+7) + 3 = 13$
- Since $f(G) < f(H)$, so it decides to go to node G.

- **Path- A $\rightarrow$ F $\rightarrow$ G**

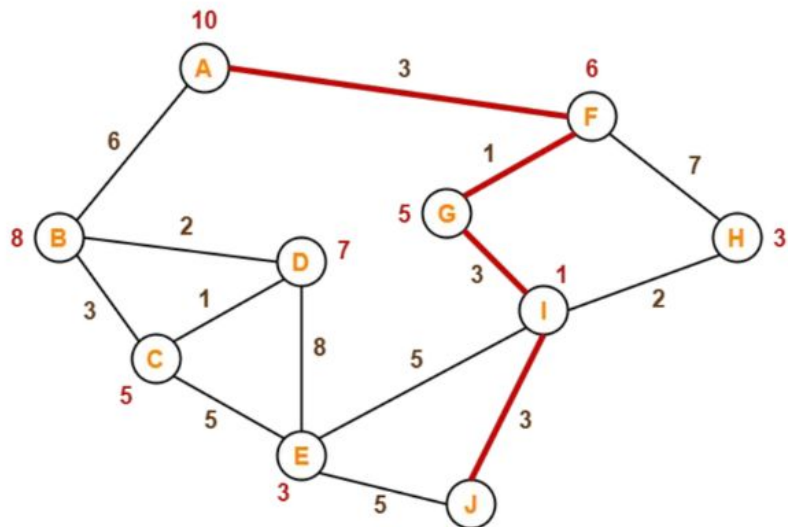


- **Step-03:**

- Node I can be reached from node G.
- A* Algorithm calculates $f(I)$.
- $f(I) = (3+1+3) + 1 = 8$
- It decides to go to node I.
- **Path- A → F → G → I**

- **Step-04:**

- Node E, Node H and Node J can be reached from node I.
- A* Algorithm calculates $f(E)$, $f(H)$ and $f(J)$.
- $f(E) = (3+1+3+5) + 3 = 15$
- $f(H) = (3+1+3+2) + 3 = 12$
- $f(J) = (3+1+3+3) + 0 = 10$
- Since $f(J)$ is least, so it decides to go to node J.
- **Path- A → F → G → I → J**



A* ALGORITHM

2	8	3
1	6	4
7		5

Initial State

1	2	3
8		4
7	6	5

Final State

Initial State

2	8	3
1	6	4
7		5

$$\begin{aligned}g &= 0 \\h &= 4 \\f &= 0+4 = 4\end{aligned}$$

$$f(n) = g(n) + h(n)$$

Consider $g(n)$ = Depth of node and $h(n)$ = Number of misplaced tiles.

A* ALGORITHM

1	2	3
8		4
7	6	5

Final State

Initial State

2	8	3
1	6	4
7		5

$$\begin{aligned}g &= 0 \\h &= 4 \\f &= 0+4 = 4\end{aligned}$$

2	8	3
1	6	4
	7	5

$$\begin{aligned}g &= 1 \\h &= 5 \\f &= 1+5 = 6\end{aligned}$$

2	8	3
1		4
7	6	5

$$\begin{aligned}g &= 1 \\h &= 3 \\f &= 1+3 = 4\end{aligned}$$

2	8	3
1	6	4
7	5	

$$\begin{aligned}g &= 1 \\h &= 5 \\f &= 1+5 = 6\end{aligned}$$

A* Algorithm

Advantages

- Guarantees optimal (shortest) path with admissible heuristic
- Complete: finds a solution if one exists
- More efficient than BFS with good heuristics
- Combines benefits of Uniform Cost Search and Greedy Best-First Search
- Widely used in pathfinding, robotics, and game AI

Disadvantages

- Very high memory usage
- Performance highly depends on heuristic quality
- Computationally expensive for large state spaces
- Not suitable for very deep or complex problems without optimization
- Requires admissible heuristic to maintain optimality

Constraint Satisfaction Problem

A Constraint Satisfaction Problem (CSP) is defined by:

- **Variables:** $X = \{X_1, X_2, \dots, X_n\}$
- **Domains:** D_i = possible values for variable X_i
- **Constraints:** Restrictions on allowable combinations of values
- **Goal:**
 - i. Assign a value to each variable such that all constraints are satisfied
 - ii. CSPs are search problems with constraints instead of costs

Example 1: Exam Scheduling (CSP)

Variables:

- Math, Physics, Chemistry

Domains:

- {Mon, Tue, Wed}

Constraints:

- Math $\neq$ Physics
- Chemistry $\neq$ Wednesday

One valid solution:

- Math = Monday
- Physics = Tuesday
- Chemistry = Monday

Example 2: 8-Puzzle Game (CSP)

Variables (X):

- Each tile position in the grid
- $X=\{P1,P2,...,P9\}$ (where one position contains the blank)

Domains (D):

- Possible tile values
- $D=\{1,2,3,4,5,6,7,8,_ \}$

Constraints (C):

- **Uniqueness Constraint:**
- Each tile value appears exactly once in the grid
- **Move Constraint:**
- Only tiles adjacent to the blank can be moved
- **State Validity Constraint:**
- Only configurations reachable via legal moves are allowed

Goal Constraint: Needed configuration

Example 3: Sudoku (Most famous CSP)

Variables:

- Each empty cell. (9X9)

Domain:

- Numbers 1–9.

Constraints:

- No same number in row
- No same number in column
- No same number in 3×3 box